

Kit 1 - Sprint 1



Projet smartEngine

Prédiction de churn pour un SaaS B2B

Modules

Scoring prédictif et Marketing

Ciblage et Data Marketing

Préparé par Nejma HAMMOUTOU

1. Contexte client

1.1 La société RavenStack

RavenStack est une entreprise SaaS B2B qui commercialise une plateforme de gestion de projets à destination des équipes tech. **SaaS (Software as a Service)** désigne un logiciel distribué via abonnement sur internet, sans installation locale. RavenStack opère avec trois niveaux d'offre : Starter, Growth et Enterprise.

Comme beaucoup d'acteurs SaaS, RavenStack fait face à un problème de **churn** : une partie de ses clients résilie son abonnement chaque mois. Le churn désigne le taux d'attrition client, c'est-à-dire la proportion de clients perdus sur une période donnée. Chaque résiliation génère une perte de **MRR** (Monthly Recurring Revenue : le chiffre d'affaires mensuel prévisible tiré des abonnements actifs), difficile à anticiper sans système de prédiction.

1.2 Votre mission

RavenStack vous mandate pour concevoir **smartEngine** (par défaut), un système intelligent de prédiction de churn. Votre mission couvre l'ensemble de la chaîne de valeur data :

- Analyser et nettoyer les données clients disponibles
- Identifier les signaux comportementaux précurseurs du churn
- Construire un modèle de scoring prédictif
- Déployer ce scoring dans un dashboard interactif à destination des équipes **Customer Success** : les collaborateurs chargés de la relation client après la vente, dont la mission est de s'assurer que les clients utilisent bien le produit et renouvellent leur abonnement
- Automatiser des alertes pour les comptes à risque élevé

Tache de recherche

Le scoring prédictif appliqué à des données clients soulève des questions réglementaires. Avant de commencer le projet, documentez les points suivants dans votre base de connaissances personnelle :
Qu'est-ce que le RGPD ? Quels sont ses grands principes (licéité, minimisation, limitation de conservation) ?

L'article 22 du RGPD encadre les décisions automatisées, y compris le profilage. Qu'impose-t-il concrètement ? Comment cela s'applique-t-il à un score de churn utilisé pour prioriser les clients à contacter ?

Qu'est-ce que la loi Informatique et Libertés ? En quoi complète-t-elle le RGPD sur le territoire français ?

Ces notions devront apparaître dans votre dossier de conception (Sprint 1 : section cadrage du projet).

1.3 Une approche par agents IA

Sur ce projet, vous ne codez pas vous-mêmes. Vous rédigez des agents, des fichiers texte qui décrivent une mission précise, et c'est l'outil d'orchestration qui les exécute : il lit vos instructions, génère le code Python, l'exécute et produit les résultats.

Votre rôle est de diriger, pas de coder. Vous définissez ce que l'agent doit faire, vous vérifiez ce qu'il produit, vous itérez si le résultat ne correspond pas à l'attendu.

Dans ce premier sprint, vous installez l'outil d'orchestration, vous rédigez le fichier GEMINI.md qui donne le contexte permanent du projet. A partir du Sprint 2, vous rédigerez des agents spécialisés (fichiers .md dans .gemini/agents/) qui rendent vos tâches réitérables et partageables entre membres de l'équipe.

Le guide **Orchestration d'agents IA** fourni avec ce kit explique cette approche en détail. Lisez-le avant de commencer.

1.4 Le dataset

RavenStack vous fournit un dataset synthétique représentatif de ses données réelles, composé de 5 fichiers CSV interconnectés :

[kaggle | SaaS subscription and churn analytics dataset](#)

Fichier	Contenu	Volume indicatif
accounts.csv	Données des comptes clients (secteur, taille, plan)	~5 000 lignes
subscriptions.csv	Historique des abonnements et modifications	~5 000 lignes
feature_usage.csv	Utilisation mensuelle des fonctionnalités	~60 000 lignes
support_tickets.csv	Tickets de support ouverts par les comptes	~15 000 lignes
churn_events.csv	Dates et motifs de résiliation constatés	~1 200 lignes

2. Organisation de l'équipe

2.1 Constitution des groupes

Vous travaillez en groupes de 4 étudiants selon la méthodologie **Scrum**, un cadre de travail itératif découpé en **sprints** (périodes de travail courtes avec des livrables définis). Chaque groupe constitue une équipe évaluée collectivement sur ses livrables.

Pour comprendre la méthode Scrum : [La méthode SCRUM pour les nuls \(Agiliste.fr\)](https://agiliste.fr/)

Lors de la première séance, chaque groupe désigne :

- **Un Product Owner** : il incarne la voix du client. Il s'assure que ce que l'équipe produit répond bien au besoin de l'entreprise, il gère le **backlog** et il arbitre les priorités
- **Un Scrum Master** : il facilite le travail de l'équipe. Il anime les **daily standups**, surveille les blocages, s'assure que les livrables sont rendus dans les délais et que les rituels Scrum sont respectés
- **Deux développeurs IA** : ils rédigent, testent et itèrent sur les agents de l'outil d'orchestration qui produisent le code et les analyses du projet

Rotation des rôles

Les rôles Product Owner et Scrum Master tournent à chaque sprint. Chaque étudiant doit avoir occupé chaque rôle au moins une fois sur l'ensemble du projet.

2.2 Outils de gestion de projet

Pour le suivi du backlog, vous choisissez librement votre outil parmi les solutions suivantes, ou toute autre solution que vous justifiez dans votre dossier de conception : GitHub Projects, Notion, Trello ou Linear.

Quel que soit l'outil choisi, le backlog doit être accessible en lecture depuis un lien que vous me transmettez. Chaque tâche doit indiquer son intitulé, le responsable, le sprint concerné et son statut.

2.3 Planning du projet

Le projet se déroule sur 4 sprints repartis entre mars et mai 2026.

Sprint	Dates	Heures	Thème
Sprint 1	9-11 mars	10h	Découverte et mise en place
Sprint 2	30 mars	6h	Traitement des données
Sprint 3	27 avril	8h	Modélisation
Sprint 4	26-29 mai	10h	Déploiement et maintenance

Entre chaque sprint, vous continuez le travail en autonomie. Les daily standups se tiennent à chaque séance. La sprint review a lieu à la fin de chaque bloc de séances.

3. Infrastructure technique

3.1 Dépôt GitHub

Chaque groupe crée un dépôt GitHub privé pour l'ensemble du projet. Ce dépôt sera le point central de tout votre travail : code, agents, rapports et dossier de conception.

Si vous n'avez jamais utilisé Git ni GitHub : [Introduction a Git et GitHub](#)

Creation du dépôt (Scrum Master)

- Sur GitHub, cliquer sur 'New repository'
- Nommer le dépôt : smartengine-groupe-[numero]
- Choisir 'Private' et cocher 'Add a README file'
- Dans Settings > Collaborators, inviter les 3 autres membres et moi-même

Récupération du dépôt (autres membres)

```
# dans un terminal sur votre pc
cd Documents
# clone du repo en local
git clone [url]
cd smartengine-groupe-[numero]
```

Flux de travail par branches

Une branche est une copie indépendante du projet sur laquelle vous travaillez sans affecter le travail des autres. La branche **main** est la version officielle et stable du projet. Chaque membre travaille sur sa propre branche et y intègre ses modifications. A la fin du sprint, le travail valide est fusionné dans **main**.

Pour comprendre le concept de branches : [Git branches - documentation officielle](#)

Après le clone, chaque membre crée une branche à son prénom. Vous ne travaillez jamais directement sur la branche **main**.

```
# Créer et basculer sur votre branche personnelle
git checkout -b votre-prenom
```

A chaque sprint, le membre en charge du développement fusionne sa branche dans main une fois les livrables valides :

```
git checkout main
git pull
git merge prenom
git push origin main
```

Structure du dépôt à la fin du Sprint 1

```
smartengine-groupe-X/
|-- .gitignore          # data/raw/ et *.ipynb exclus du suivi Git
|-- .gemini/
|   |-- agents/        # vos agents IA (à partir du Sprint 2)
|-- data/
|   |-- raw/           # les 5 CSV RavenStack (ne jamais modifier)
|-- outputs/           # rapports générés par les agents
|-- src/               # scripts Python
|-- docs/
|   |-- standups/      # comptes-rendus des daily standups
```

```
| `-- ...           # livrables du kit
|-- GEMINI.md
`-- README.md
```

Le .gitignore

Un fichier .gitignore place à la racine de votre dépôt indique à Git quels fichiers ignorer. Les fichiers listés dedans ne seront jamais commités, même si vous faites git add .

Les données brutes et les notebooks individuels ne doivent pas être commités. Chaque membre récupère les CSV (depuis discord #datasets) et les place localement dans data/raw/. Contenu minimal du .gitignore :

```
data/raw/
*.ipynb
docs/exploration-*/
```

3.2 Installation de l'outil d'orchestration

L'outil d'orchestration vous permet d'interagir avec un agent IA directement depuis votre terminal, avec la capacité de lire, créer et modifier des fichiers de manière autonome. Site officiel :

geminicli.com

Prérequis

- Node.js version 18 ou supérieure (vérifiez avec : `node --version`). [Qu'est-ce que Node.js ?](#)
- Un compte Google
- Git installé sur votre machine

Installation

```
npm install -g @google/gemini-cli
gemini
```

Au premier lancement, votre navigateur s'ouvre automatiquement pour vous connecter avec votre compte Google. Une fois l'authentification terminée, vous êtes prêt à utiliser l'outil. Pour vérifier l'installation ultérieurement : `gemini --version`

3.3 Le fichier GEMINI.md

L'outil d'orchestration n'a aucune mémoire entre deux sessions. Le fichier GEMINI.md, placé à la racine de votre dépôt, est lu automatiquement à chaque démarrage. Tout ce que vous y écrivez est connu de l'agent sans que vous ayez besoin de le préciser : conventions de nommage, fichiers à ne pas toucher, langue des rapports, sprint en cours.

Documentation complète : [GEMINI.md \(Gemini CLI\)](#)

Contenu minimal pour ce premier sprint :

```
# Projet smartEngine - Groupe [X]

## Contexte
Nous construisons smartEngine, un système de prédiction de churn
pour RavenStack, un SaaS B2B. Les données sont dans /data/raw/.

## Conventions
```

```
- Scripts Python -> /src/  
- Rapports      -> /outputs/  
- Ne jamais modifier /data/raw/  
- Tous les rapports sont en francais  
  
## Sprint en cours  
Sprint 1 - Decouverte et mise en place
```

4. Le dossier de conception

Le **dossier de conception** est le document central du projet. Il retrace toutes les décisions prises par votre équipe au fil des sprints : pourquoi vous avez choisi tel outil, comment vous avez traité les données, quel algorithme vous avez retenu et sur quelle base.

Ce n'est pas un compte-rendu de ce que vous avez fait. C'est la justification de pourquoi vous l'avez fait. Il est rédigé en parallèle du projet, sprint par sprint, et conservé dans docs/dossier-conception.docx.

4.1 Exemple de structure minimale

Sprint	Section
Sprint 1	1. Cadrage du projet (contexte métier, objectifs, contraintes RGPD, choix d'outils)
Sprint 2	2. Traitement des données (nettoyage, qualité, feature engineering)
Sprint 3	3. Modélisation (choix d'algorithmes, métriques, biais et éthique)
Sprint 4	4. Déploiement et recommandations (dashboard, data storytelling, recommandations stratégiques)

Livable

Première partie du dossier de conception

docs/dossier-conception.docx

Chaque choix technique est justifié, pas seulement nommé

5. Travaux du Sprint 1

5.1 Veille sur les outils

Avant de produire la moindre ligne de code, vous réalisez une veille structurée sur les outils de la stack.

La stack technique

Le projet repose sur une stack technique imposée. Pour chacun des outils ci-dessous, vous documentez son rôle dans le projet, ses avantages, ses limites et ses alternatives. C'est ce que vous produisez dans le rapport de veille.

- **Gemini CLI** : outil en ligne de commande permettant d'interagir avec un agent IA depuis le terminal
- **Python / pandas** : langage de programmation et bibliothèque de manipulation de données
- **scikit-learn** : bibliothèque Python de machine learning
- **Streamlit** : framework Python pour créer des interfaces web
- **n8n** : outil d'automatisation de workflows, open source et auto-hebergeable
- **GitHub** : plateforme de gestion de code source et de collaboration

Tache de recherche

Qu'est-ce que le churn rate ? Comment se calcule-t-il ?

Quel impact le churn a-t-il sur le MRR d'un éditeur SaaS ?

Quels sont les benchmarks de churn acceptables dans le secteur SaaS B2B ?

A consigner dans votre base de connaissances personnelle.

Livrable

Rapport de veille outils

docs/veille-outils.md

Fiche pour chacun des 6 outils

Structure par fiche : Présentation - Rôle dans le projet - Avantages - Limites - Alternatives

Sources citées avec liens actifs

5.2 Reformulation du brief client

Vous reformulez le brief client dans vos propres mots.

Le brief doit permettre à une personne qui ne connaît pas le projet de comprendre immédiatement qui est le client, quel est son problème et ce que vous allez construire.

Tache de recherche

Pour reformuler le brief de manière pertinente, vous devez comprendre le modèle économique de votre client. Documentez :

Qu'est-ce que le modèle SaaS B2B ? Quelles sont les métriques clés ?

Quelles sont les étapes du parcours client SaaS ? A quelle étape se situe le problème de churn ?

Qu'est-ce qu'une équipe Customer Success ? Quel est son rôle dans la rétention ?

Quels sont les canaux de communication typiques d'un éditeur SaaS B2B pour interagir avec ses clients ?

Livable

Brief client reformulé

docs/brief-client.md

Inclut une section 'Critères de succès' avec au moins 3 critères mesurables

5.3 Votre premier agent

Avant de découvrir le dataset en équipe, chaque membre de l'équipe crée individuellement son propre agent de découverte des données. Vous apprenez à écrire un agent, vous l'exécutez, vous observez ce qu'il produit.

Créez d'abord un dossier personnel à votre prénom dans docs/ :

docs/exploration-prenom/

Enfin, récupérer le guide "**Orchestration d'agents IA**" dans discord #liens-outils.

Livable

Agent d'exploration du dataset.

5.4 Découverte du dataset

Lancez l'agent que vous avez créé précédemment. Demandez-lui d'explorer chaque fichier CSV dans data/raw/. Pour chacun des 5 fichiers, relevez :

- Le nombre de lignes et de colonnes
- Le nom et le type de chaque colonne
- Les 3 à 5 premières lignes
- Les valeurs qui semblent manquantes ou incohérentes
- Les colonnes que vous pensez utiles pour prédire le churn, et pourquoi

L'outil d'orchestration va générer et exécuter du code Python pour répondre à vos questions. Votre travail est de vérifier les résultats et de les synthétiser dans la fiche de découverte.

Tâche de recherche

L'utilisation de données pour prédire le comportement des clients soulève des questions éthiques. Documentez :

Qu'appelle-t-on un biais algorithmique ? Donnez un exemple concret dans un contexte marketing.

Qu'est-ce que la sobriété numérique ? Comment se traduit-elle dans un projet data (choix des données collectées, volume de stockage, fréquence de traitement) ?

Qu'est-ce que la transparence algorithmique ? Pourquoi est-elle importante quand un score influence des décisions commerciales ?

Livable

Notebook de découverte du dataset (individuel)

- docs/exploration-prenom/notebooks/decouverte-dataset.ipynb

Fiche de découverte du dataset (individuel)

- docs/exploration-prenom/decouverte-dataset.md

➔ Fich collective, un tableau récapitulatif par CSV : Nom, Nombre de lignes, Colonnes clés, Observations

Section finale : 5 questions métier à explorer avec l'agent

5.5 Mise en place du backlog Sprint 1

Chaque tâche du backlog est une user story. [Guide des user stories \(Atlassian\)](#)

En tant que [role], je veux [action] afin de [bénéfice].

Exemples :

- En tant que Scrum Master, je veux créer le dépôt GitHub et inviter les membres afin que tout le monde puisse contribuer.
- En tant que Product Owner, je veux reformuler le brief client afin que l'équipe comprenne les objectifs métier de RavenStack.

Livable

Backlog Sprint 1

Outil au choix (GitHub Projects, Notion, Trello, Linear)

Au moins une user story par livrable du sprint

Chaque story assignée à un membre de l'équipe

6. Livrables

Livrable	Fichier attendu	Responsable
Depot GitHub structure	README.md + arborescence complète	Scrum Master
Rapport de veille outils	docs/veille-outils.md	Toute l'équipe
Brief client reformule	docs/brief-client.md	Product Owner
Première partie du dossier de conception	docs/dossier-conception.docx	Toute l'équipe
Agent d'exploration	.gemini/agents/data-explorer.md	Toute l'équipe
Fiche de découverte dataset + .ipynb	docs/ exploration-prenom/decouverte-dataset.md	Toute l'équipe
Fiche collective d'exploration	docs/decouverte-dataset.md	Scrum Master
GEMINI.md initial	GEMINI.md (racine)	Scrum Master
Backlog Sprint 1	Lien outil de gestion	Scrum Master

7. Rituels Scrum

7.1 Daily Standup

Chaque journee de cours commence par un daily standup de 10 minutes maximum. Chaque membre repond a trois questions :

- Qu'ai-je accompli depuis le dernier standup ?
- Que vais-je accomplir aujourd'hui ?
- Y a-t-il un obstacle qui m'empeche d'avancer ?

Le compte-rendu est depose dans docs/standups/AAAA-MM-JJ.md a la fin de chaque journee.

7.2 Sprint Review

A la fin de chaque sprint, je passe dans chaque groupe pour une revue de 10 minutes. Vous partagez votre ecran et montrez ce que vous avez produit. Le compte-rendu suit ce format :

- Ce que nous avons prevu de faire
- Ce que nous avons reellement fait
- Ce que nous avons appris
- Ce que nous ferons differemment au sprint suivant

8. Ressources

Ressource	Lien
Guide Orchestration d'agents IA	Discord #liens-outils
Documentation Gemini CLI	geminicli.com/docs/
Sous-agents Gemini CLI	geminicli.com/docs/core/subagents/
Repo de reference agents	github.com/contains-studio/agents
Introduction a Git et GitHub	docs.github.com/fr/get-started
Guide Scrum officiel	scrum.org/resources/scrum-guide
Guide des user stories	atlassian.com/fr/agile/user-stories
Dataset RavenStack	Discord #datasets